Padding Oracle Attack Lab

Module: Advanced Cryptography
Master's Program: Cryptography and Information Security
Mohammed V University in Rabat – Faculty of Sciences

May 2026

1 Overview and Objectives

The learning objective of this lab is to gain hands-on experience with an interesting attack on crypto systems known as the **Padding Oracle Attack**.

In this lab, two oracle servers running inside a container. Each oracle has a secret message hidden inside, and it lets you know the ciphertext and the IV, but not the plaintext or the encryption key. You can send a ciphertext (and the IV) to the oracle; it will decrypt the ciphertext using the encryption key, and tell you whether the padding is valid or not. Your job is to use the response from the oracle to figure out the content of the secret message.Some systems, when decrypting a ciphertext, verify whether the padding is valid or not and throw an error if it is invalid. This behavior enables an attacker to figure out the content of a secret message without knowing the encryption key or the plaintext.

This lab covers:

- Secret-key encryption (AES-CBC).
- Encryption modes and PKCS#7 padding schemes.
- Side-channel attacks via oracle responses.

2 Lab Environment Setup

This lab is conducted on the **SEED Ubuntu 20.04 VM**. Unlike other labs, we use a containerized environment to run the padding oracle servers.

2.1 Downloading Labsetup

The environment is set up by downloading the `Labsetup.zip` file from the lab's website. After unzipping, the `docker-compose.yml` file is used to configure the containers.

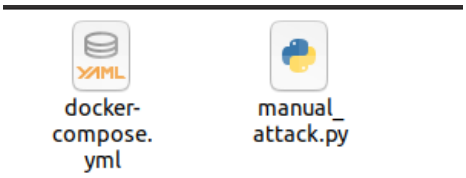


Figure 1: Labsetup file

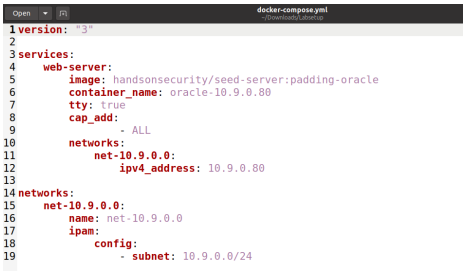


Figure 2: The docker-compose.yml file

2.2 Docker Commands and Aliases

The SEED VM provides aliases for commonly used Docker commands to simplify the management of the lab containers 3 .

```
cd Labsetup
```

All the containers will be running in the background. To run commands on

Alias	Docker Command Equivalent
dcbuild	docker-compose build
dcup	docker-compose up
dedown	docker-compose down
dockps	docker ps
docksh <id>	docker exec -it <id> /bin/bash

Table 1: Docker aliases in the SEED VM environment.

a container, we often need to get a shell on that container4. We first need to use the "docker ps" command to find out the ID of the container, and then use "docker exec" to start a shell on that container.

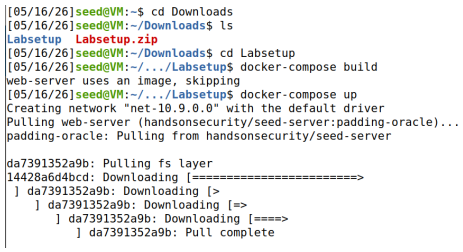


Figure 3: Verification of the running containers using the `dcup` or `dockps` command.

Listing 1: Docker helper commands example

```
$ dockps      # Alias for: docker ps --format "{{.ID}} {{.Names}}"
$ docksh <id>  # Alias for: docker exec -it <id> /bin/bash
# The following example shows how to get a shell inside hostC
$ dockps
b1004832e275 hostA-10.9.0.5
0af4ea7a3e2e hostB-10.9.0.6
9652715c8e0a hostC-10.9.0.7

$ docksh 96
root@9652715c8e0a:/#
```

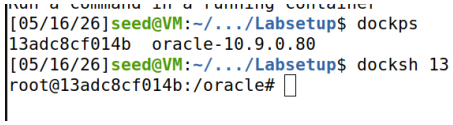


Figure 4: Container ID

The first 12 characters on the left Listing 1 (e.g., b1004832e275 or 0af4ea7a3e2e) represent your unique Container ID.You do not need to type the entire ID string. Typing just the first 2 or 3 characters is sufficient, as long as they are unique among all running containers.

3 Task 1: Getting Familiar with Padding

For block ciphers like AES, padding is required when the plaintext size is not a multiple of the block size. This lab utilizes the **PKCS#7** padding scheme.

3.1 Experimentation

We use the `-nopad` option in OpenSSL to disable automatic padding removal during decryption, allowing us to see the padded data.

Step 1: Create the Three Test Files 5: Use the `echo -n` command to create three separate files with exactly 5 bytes, 10 bytes, and 16 bytes respec-

tively. The `-n` flag is crucial because it prevents a newline character from being added to the end of the file.

Listing 2: Create Three files

```
echo -n "12345" > P5
echo -n "1234567890" > P10
echo -n "1234567890123456" > P16
```

Step 2: Encrypt the Files Using `AES-128-CBC` 6:encrypt each file using the `OpenSSL` command line tool with a password or key of your choice. For simplicity during lab testing, you can use a basic password via the `-k` flag:

Listing 3: Encrypt the files

```
openssl enc -aes-128-cbc -e -in P5 -out C5 -k password
openssl enc -aes-128-cbc -e -in P10 -out C10 -k password
openssl enc -aes-128-cbc -e -in P16 -out C16 -k password
```

Step 3: Decrypt with Padding Removal Disabled (-nopad) 7:normally, OpenSSL automatically strips away the padding bytes during decryption so the user only sees the original text. To inspect the raw padding bytes added by the system, decrypt the ciphertexts using the `-nopad` option:

Listing 4: Decrypt the files

```
openssl enc -aes-128-cbc -d -nopad -in C5 -out P5_padded
    -k password
openssl enc -aes-128-cbc -d -nopad -in C10 -out P10_padded
    -k password
openssl enc -aes-128-cbc -d -nopad -in C16 -out P16_padded
    -k password
```

Step 4: Analyze the Padding with a Hex Tool 8:because the added padding bytes are non-printable characters, you must view the decrypted result using a hex tool like `xxd` to verify the `PKCS#7` format structure:

Listing 5: Decrypt the files

```
xxd P5_padded
xxd P10_padded
xxd P16_padded
```

Using the method above, please figure out what paddings are added to the three files. When decrypting the 16 byte file, why do we see a full block of padding? Why is this necessary?

4 Task 2: Padding Oracle Attack (Level 1)

Some systems, when decrypting a given ciphertext, verify whether the padding is valid or not, and throw an error if the padding is invalid. This seemly-harmless behavior enables a type of attack called padding oracle attack. The attack was originally published in 2002 by Serge Vaudenay, and many well-known systems were found vulnerable to this type of attacks, including Ruby on Rails, ASP.NET, and OpenSSL.

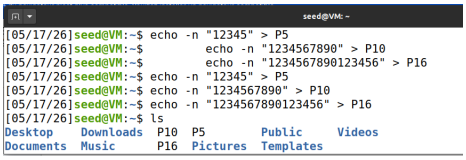


Figure 5: Step 1

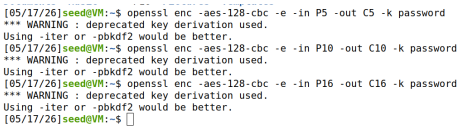


Figure 6: Step 2

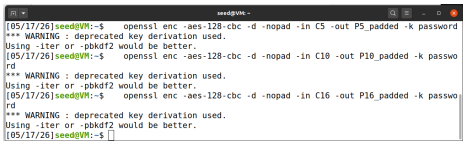


Figure 7: Step 3

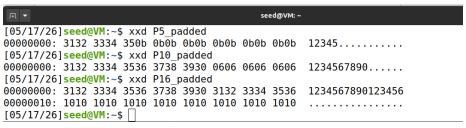


Figure 8: Visualizing padding for 5-byte, 10-byte, and 16-byte files.

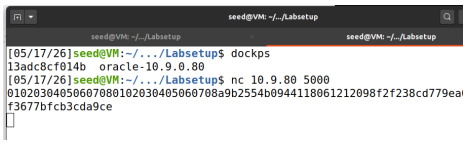


Figure 9: Netcat Tool.

4.1 Oracle Setup

In this task, we provide a padding oracle hosted on port 5000. The oracle has a secret message inside,and it prints out the ciphertext of this secret message. The encryption algorithm and mode used is AES-CBC, and the encryption key is K, which is unknown to others.

1. open your terminal inside the SEED VM and move to the directory where your lab configuration files are stored:`cd /Desktop/Labsetup`
2. To start the oracle servers in the background, run the pre-configured SEED environment shortcut:`dcup`
3. The Level 1 oracle server runs inside a dedicated container and listens on port 5000. Let's verify that the container is active and retrieve its short ID: `dockps`
   (This is the shortcut for docker ps). You should see a running container named something like padding-oracle-level1. Take note of the first few characters of its ID (e.g., b1004832e275).
4. Before executing any automated or manual script, you must test if you can reach the server and receive the encrypted data. Use the Netcat tool (nc) to connect to the oracle's designated IP address and port: `nc 10.9.0.80 5000`
   As soon as you execute the nc command, the oracle server will respond by printing a long, continuous hexadecimal string to your terminal window:

   - The first 32 hex characters (16 bytes): This represents the Initialization Vector (IV) generated by the server.
   - The remaining hex characters (32 bytes / 2 blocks): This is the Ciphertext containing Alice's hidden secret message.

The oracle accepts input from you. The format of the input is the same as the message above: 16-bytes of the IV, concatenated by the ciphertext. The oracle will decrypt the ciphertext using its own secret key K and the IV provided by you. It will not tell you the plaintext, but it does tell you whether the padding is valid or not. Your task is to use the information provided by the oracle to figure out the actual content of the secret message.

4.2 The Core Logic of the Attack

The objective of this task is to figure out the plaintext of the secret message. It has two blocks P1 and P2. We only need to get P2 (getting P1 is similar).The oracle accepts an input format consisting of IV + Ciphertext. The ciphertext has two blocks (C_1 and C_2). In this level, we are targeting the second block of plaintext (P_2).

As shown in the CBC decryption diagram, the block cipher decrypts C_2 to produce an intermediate mathematical value called D_2. The final plaintext is then calculated via a simple XOR operation:

$$P_2 = C_1 \oplus D_2$$

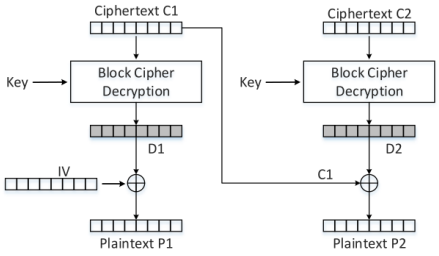


Figure 10: Decryption Using CBC.

We can compromise this by replacing the original C_1 block with a custom block we control, called CC_1. When we send IV + CC1 + C2 to the oracle, it decrypts C_2 into D_2 and checks if the padding of the result is structurally valid.

4.3 Executing the Manual Attack Step-by-Step

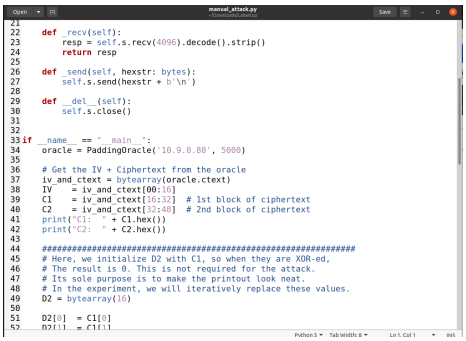
1. Open the skeleton file manual_attack.py 11using your preferred editor (e.g., gedit manual_attack.py or nano manual_attack.py). Inside, you will see a loop that tests 256 possible byte values.

2. **Round 1: Finding the Last Byte ($D_2[15]$):**

   To find the very last byte of D_2 ($D_2[15]$), we want the oracle's internal decryption to result in a valid PKCS#7 padding of 0x01.

   - Set your target index variable K = 1 in the script.
   - Run the script: `python3 manual_attack.py`
   - Look closely at the terminal output 12. The loop will test all numbers from 0 to 255 until it hits a value that returns a "Valid" status.
   - And Because you know that this specific byte i resulted in a valid padding value of `0x01`, you can derive the intermediate value $D_2[15]$:

$$D_2[15] = i \oplus 0x01$$



```
    def _recv(self):
        resp = self.s.recv(4096).decode().strip()
        return resp

    def _send(self, hexstr: bytes):
        self.s.send(hexstr + b'\n')

    def __del__(self):
        self.s.close()


if __name__ == "__main__":
    oracle = PaddingOracle('10.9.0.80', 5000)

    # Get the IV + Ciphertext from the oracle
    iv_and_ctext = bytearray(oracle.ctext)
    IV    = iv_and_ctext[00:16]
    C1    = iv_and_ctext[16:32]  # 1st block of ciphertext
    C2    = iv_and_ctext[32:48]  # 2nd block of ciphertext
    print("C1:  " + C1.hex())
    print("C2:  " + C2.hex())

    ################################################################
    # Here, we initialize D2 with C1, so when they are XOR-ed,
    # The result is 0. This is not required for the attack.
    # Its sole purpose is to make the printout look neat.
    # In the experiment, we will iteratively replace these values.
    D2 = bytearray(16)

    D2[0]  = C1[0]
```

Figure 11: Manual Attack File.

```
[05/17/26]seed@VM:~/.../Labsetup$ python3  manual_attack.py
C1:  a9b2554b09441180612120 98f2f238cd
C2:  779ea0aae3d9d020f3677bfcb3cda9ce
Valid: i = 0xcf
CC1: 000000000000000000000000000000cf
P2:  00000000000000000000000000000000
[05/17/26]seed@VM:~/.../Labsetup$
```

Figure 12: the single value of i that triggered the Valid response

```
106
107    # Once you get all the 16 bytes of D2, you can easily get P2
108    P2 = xor(C1, D2)
109    print("P2:  " + P2.hex())
```

Figure 13: The plaintext P2.

3. **Round 2: Finding the Next Byte ($D_2[14]$) :**

   Now that you know the true value of $D_2[15]$, you can move backward to find $D_2[14]$. For this round, we want the oracle to see a valid two-byte padding of 0x02.

   - Update your script's configuration: change K = 2.
   - You must now hardcode the position of $CC_1[15]$ so that it forces the underlying decryption to always equal 0x02. **Set:** $CC_1[15] = D_2[15] \oplus 0x02$**.**
   - Run the script again: `python3 manual_attack.py` .The loop will run through all 256 values for $CC_1[14]$ until it outputs a new Valid value for i.

You can use the skeleton code as your basis, manually change the value of K, use the execution result in each round to set the $D2$ array accordingly, and then re-run the program with an updated $CC1$ to get the next byte of $D2$, i.e., $D2[14]$. Repeating this step, you can sequentially derive $D2[13], D2[12], \ldots, D2[0]$. Once you obtain the entire $D2$ array, the final value of the plaintext block $P2$ can be easily computed using the relation $P2 = C1 \oplus D2$ 13.

5 Task 3: Automated Attack (Level 2)

the objective changes from manual exploitation to full automation. Instead of updating K and hardcoding the discovered bytes of D_2 one by one, you will write a nested loop script to automatically decrypt all blocks of the plaintext in a single run.

5.1 The Automation Logic

To automate the attack across an entire block, your script must execute two main layers of loops:

- The Outer Loop (Index K): Moves backward from the last byte of the block (index 15 down to 0).
- The Inner Loop (Value i): Iterates through all 256 possible byte values (0x00 to 0xff) for the current position.

Before brute-forcing the current byte at position $16 - K$, the script must pre-calculate and set all the trailing bytes in CC_1 that you have already discovered. This ensures they align perfectly with the target padding structure (K) expected by the oracle.

5.2 Automated Script (automated_attack.py)

Create a new file named `automated_attack.py` inside your Labsetup directory and use this code, Listing 6.

1. Ensure your containers are actively running: `dcup`
2. Run your newly created automation script: `python3 automated_attack.py`

Listing 6: Automated Padding Oracle Attack Script (automated_attack.py)

```
#!/usr/bin/python3
import socket
from binascii import hexlify, unhexlify

class PaddingOracle:
    def __init__(self, host, port) -> None:
        self.s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.s.connect((host, port))
        # Receive the initial ciphertext configuration sent upon
            connection
        ciphertext = self.s.recv(4096).decode().strip()
        self.ctext = unhexlify(ciphertext)

    def decrypt(self, ctext: bytes) -> str:
        self._send(hexlify(ctext))
        return self._recv()

    def _recv(self):
```

```
        return self.s.recv(4096).decode().strip()

    def _send(self, hexstr: bytes):
        self.s.send(hexstr + b'\n')

    def __del__(self):
        self.s.close()

if __name__ == "__main__":
    # Level-2 server configuration
    oracle = PaddingOracle('10.9.0.80', 6000)
    iv_and_ctext = bytearray(oracle.ctext)

    # Extract structural blocks (16 bytes each)
    IV = iv_and_ctext[0:16]
    C1 = iv_and_ctext[16:32]
    C2 = iv_and_ctext[32:48]

    # Data structures to record intermediate targets
    D2 = bytearray(16)
    CC1 = bytearray(16)
    P2 = bytearray(16)

    print("[+] Starting automated Padding Oracle Attack on Block 2...")

    # Loop through each byte index from right to left (K from 1 to 16)
    for K in range(1, 17):
        idx = 16 - K  # Current array position index
        print(f"[*] Brute-forcing index {idx} (Target padding value: 0x{K:02x})...")

        # Dynamically set trailing bytes based on previous D2 discoveries
        for trailing_idx in range(idx + 1, 16):
            CC1[trailing_idx] = D2[trailing_idx] ^ K

        # Iterate through all 256 possible options for the target position
        for i in range(256):
            CC1[idx] = i
            status = oracle.decrypt(IV + CC1 + C2)

            if status == "Valid":
                # Save intermediate decryption state byte
                D2[idx] = i ^ K
                # Compute plaintext byte via original ciphertext dependency
                P2[idx] = C1[idx] ^ D2[idx]
                print(f"    -> Valid Match Found! D2[{idx}] = 0x{D2[idx]:02x} | P2[{idx}] = {repr(chr(P2[idx]))}")
                break

    print("\n[+] Full Block Decryption Completed Successfully!")
    print(f"Intermediate Data State D2 (Hex): {D2.hex()}")
```

```
67      print(f"Decrypted Plaintext Block P2 (Hex): {P2.hex()}")
68      print(f"Decrypted Secret Message String:
            {P2.decode(errors='ignore')}")
```

This Python script automates a Padding Oracle Attack, specifically tailored to exploit the Level-2 containerized oracle server running on port 6000.

The purpose of this script is to sequentially reconstruct a hidden plaintext block (P_2) without cracking the AES key. It achieves this by submitting systematically altered ciphertext blocks to the server and analyzing whether the server's validation mechanisms report a correct or corrupted padding structure

6 Conclusion

This lab demonstrates that even a single bit of information (padding validity) leaked by a server can lead to a complete compromise of confidentiality. To mitigate this, developers should ensure error messages are uniform or use authenticated encryption modes like GCM.